

NeuroForge Technology (NFT) — Overview & Architecture

Internal System Documentation & Technical Reference Manual

1. Executive Overview

NeuroForge Technology (NFT) is an advanced, multi-agent artificial intelligence coding platform designed to assist developers, teams, and organizations in automating software development workflows, conducting deep competitive and technical research, generating educational materials, and orchestrating server automations. NFT combines a responsive React frontend interface with a robust, asynchronous FastAPI backend power-house that manages memory, local search indexes, background workers, and LLM integrations.

The system utilizes advanced Retrieval-Augmented Generation (RAG) to dynamically ground agent decisions and user interactions in localized context layers: Project Files, Organization Knowledge Bases, and ephemeral User Sessions. By prioritizing user context, NFT operates with high precision, maintaining strict isolation and relevance.

2. System Architecture & Tech Stack

NFT's architecture is structured into clear layout layers that ensure modularity and scalability:

- **Frontend:** Built with React (Vite-powered) using Vanilla CSS for UI elements. It manages state transitions, parses real-time Server-Sent Events (SSE) for streaming model outputs, and handles interactive elements such as connectors configuration, workspace directory selectors, and drag-and-drop file ingestion.
- **Backend API:** Built with FastAPI. It handles routing for user authentications, organization settings, conversation logging, dynamic RAG queries, and MCP (Model Context Protocol) plugins.
- **Retrieval Engine (RAG):** Integrates a multi-layered search pipeline. ChromaDB handles vector embeddings, while a pure Python BM25 indexer performs exact keyword scoring. Reciprocal Rank Fusion (RRF) combines both results to output the most relevant chunks.
- **Database Storage:** MongoDB holds user session history, workspaces profiles, active organizations, and uploaded document metadata records.

3. Core Modules & Agent Workflows

NeuroForge is divided into five specialized core tabs, each powered by designated agent flows:

3.1. Engineer AI: Powered by LangGraph checkpointers. When a user input proposal is received, the agent starts a multi-step sequence (planning, drafting, testing, and debugging). It writes code directly to the workspace, executes compilation tests, and catches exceptions to self-repair output syntax before completion.

3.2. Conversational Chat: A general-purpose RAG interface that supports interactive chats with session memory. It integrates third-party APIs (connectors for GitHub, Gmail, and Google Drive) allowing users to retrieve code bases or draft communications dynamically.

3.3. Research AI: Chains a 5-step synchronous workflow involving a supervisor and researcher agent. It runs automated web searches, scrapes technical documentation, and compiles thorough competitive analyses into markdown reports.

3.4. Education AI: Tailored for study materials generation. It takes text inputs or uploaded course books, extracts concepts, and constructs structured summaries, quizzes, and learning roadmaps.

3.5. Automation AI: Focuses on system control. It executes pre-defined scripts, interfaces with operating system commands safely, and manages routine cron scheduling for developers.

4. Operational Flow & Team Roles

Operational workflows in NeuroForge Technology are managed across several distinct team roles to maintain high service reliability:

Role	Key Responsibilities	Active Modules
System Architect	Manages MongoDB structures, FastAPI routes, and LLM providers fallback controls.	Backend Core
AI Engineer	Builds LangGraph state loops and embeds custom agent logic parameters.	Engineer & Research
RAG Specialist	Handles document ingestion, vector embeddings math, and hybrid retrieval tuning.	Vector Storage
UI/UX Designer	Designs responsive interface styles, modals, switcher layouts, and drag-and-drop systems.	React Frontend

5. RAG Data Ingestion & Memory Layer

When a document (e.g. PDF, DOCX, XLSX, TXT) is uploaded to a temporary session or organization knowledge base:

1. The document is intercepted by `background_indexer.py`.
2. The appropriate parser in `document_processor.py` extracts text (e.g., using `pypdf` for PDF text extraction).
3. Extracted text is split into chunks of ~1000 characters with a 150-character overlap using recursive token splitting.
4. Vector representations are generated and loaded into ChromaDB, linked with metadata including `session_id` and `document_id`.
5. During a conversational sequence, user questions are condensed into search-optimized queries. Chunks from the active session are prioritized, and if confidence is low, the pipeline falls back to indexing the first chunks of the latest uploaded document from memory.